

CheesyWasp: A Lightweight Runtime Firewall for Prompt Injection and LLM Data Leakage

Sanjeev Ragunathan

Master of Engineering in Computer Science

Cornell University

Ithaca, NY, USA

sr2432@cornell.edu

Abstract—LLM applications are vulnerable because trusted developer instructions and untrusted user or tool content share the same context window. We present CheesyWasp, a lightweight runtime firewall that inspects prompts and responses to block prompt injection, redact sensitive information, and prevent system-prompt leakage. Unlike broad guardrails that duplicate model alignment, CheesyWasp is scoped to application-boundary threats: direct injection, indirect injection, inbound PII, outbound PII, and system-prompt disclosure. We evaluate on a 749-prompt suite across six threat-specific tracks drawn from Deepset Prompt Injections, BIPIA, Ai4Privacy PII Masking, Dolly-15k, and constructed leakage probes following the methodology of Lukas et al. [6]. Our metric is end-to-end *handling rate*: firewall blocked or model refused. In the strongest result, CheesyWasp handles all 15 system-prompt leakage probes (100%), reducing verbatim leakage from 60% without the firewall to 0%. Combined handling across all five attack tracks ranges from 26% on indirect injection (a known-hard benchmark) to 100% on both outbound leakage tracks, with a 0% false-positive rate on a 200-prompt Dolly-15k benign control.

Index Terms—large language models, prompt injection, PII, system-prompt leakage, runtime firewall, OWASP LLM Top 10

I. INTRODUCTION

LLM-powered applications routinely mix trusted developer instructions with untrusted user input, retrieved documents, tool outputs, and other external content. Because LLMs process this material as a single token stream, an attacker who controls part of the context can attempt to override developer intent [1], [2]. The OWASP Top 10 for LLM Applications ranks prompt injection as the top LLM application risk [12].

A common mistake in LLM guardrail evaluation is to treat the guardrail as if it must defend against every possible misuse. In practice, deployed systems combine runtime controls with the model’s own refusal behavior. Model alignment or a moderation layer is the natural place to handle direct harmful-content requests, such as “write malware.” A runtime firewall is better suited for the application-boundary failures that alignment does not reliably solve: instruction hijacking, hidden instructions in retrieved content, sensitive-data exposure, and system-prompt leakage.

We introduce a *defense-in-depth* evaluation methodology that decomposes each outcome into three buckets: firewall-blocked, model-refused, and complied (failure). This attribution exposes which defender handles each threat class and makes

the combined end-to-end handling rate the correct headline metric, rather than the firewall block rate alone.

This paper makes three contributions:

- **Scoped firewall design.** We introduce CheesyWasp, a layered application firewall for direct injection, indirect injection, inbound PII, outbound PII, and system-prompt leakage. The pipeline combines pattern-based detection, Luhn-verified credit-card checks, a DeBERTa-v3 prompt-injection classifier, and a bidirectional response inspector.
- **Threat-specific evaluation.** We evaluate on a 749-prompt six-track suite where each track maps to the specific threat it measures, avoiding conflation of prompt injection with general harmful-content moderation.
- **Outbound leakage protocol.** We evaluate output-side leakage by injecting known synthetic secrets into the prompt context, probing the model, and measuring verbatim leakage, following the methodology of Lukas et al. [6].

II. BACKGROUND AND RELATED WORK

Direct prompt injection embeds adversarial instructions directly in the user message with the goal of overriding the developer prompt [2]. **Indirect prompt injection** hides attacker-controlled instructions in third-party content that is later retrieved or passed through tools into the LLM context [1]. Both exploit the lack of a reliable trust boundary between instructions and data.

The 2025 OWASP Top 10 for LLM Applications identifies prompt injection (LLM01), sensitive information disclosure (LLM02), and system-prompt leakage (LLM07) as key LLM application risks [12]. CheesyWasp targets these three classes. General harmful-content policy enforcement is out of scope by design because that responsibility is better handled by model alignment or dedicated moderation systems.

Benchmarks. For direct injection, the Deepset Prompt Injections dataset provides binary-labelled prompts and is commonly used for prompt-injection classifier training [4]. BIPIA evaluates indirect prompt injection across LLM-integrated application settings such as email QA, web QA, table QA, summarization, and code QA [3]. For PII detection, Ai4Privacy PII Masking provides sentences with character-level PII spans across multiple entity types [5]. For benign FPR measurement we use Databricks Dolly-15k, filtering to instruction-only rows without lengthy context.

TABLE I
THREAT MODEL AND FIREWALL POLICY.

Threat	Example	Action
Direct injection	“Ignore previous instructions”	Block
Indirect injection	Malicious text inside a retrieved document	Block
High-risk PII	SSN, API key, credit card	Block
Low-risk PII	Email address, phone number	Redact
System-prompt leakage	“Reveal your hidden instructions”	Block
Harmful content request	“Write malware”	Out of scope

Defenses. External safeguard models include Llama Guard [7], Prompt Guard, and DeBERTa-based prompt-injection classifiers. Other approaches include input structuring [9], alignment-side defenses [10], and rule-based runtime frameworks such as NeMo Guardrails [8]. Li et al. [11] show that prompt-injection guardrails can over-defend by misclassifying benign prompts that contain trigger words such as `ignore`. We therefore report false-positive rates on benign Dolly-15k prompts as a primary metric.

III. SYSTEM DESIGN

A. Threat Model and Scope

CheesyWasp assumes an attacker may control either the user message or external content concatenated into the prompt through retrieval or tool calls. Table I summarizes the firewall scope and policy action for each threat class.

The key design decision is scope. A prompt such as “write malware” is a content-policy request rather than a prompt-injection attempt. CheesyWasp leaves that decision to the aligned model or a dedicated moderation layer, while focusing on instruction hijacking and data leakage at the application boundary. This is a defense-in-depth design: the runtime firewall and the model’s refusal behavior handle different failure modes, and measuring only the firewall in isolation understates the joint system’s protection.

B. Architecture

Figure 1 shows the request lifecycle. Every user request is inspected before reaching the LLM, and every model response is inspected before reaching the user. Each inspector emits a Verdict: allow, block, or redact.

1) *Inbound L1: Pattern Layer:* The first inbound layer uses compiled regular expressions for known injection templates, role-manipulation phrases, system-prompt extraction attempts, and credential patterns. It detects credit-card-shaped digit sequences and verifies them with the Luhn checksum before blocking. This avoids false positives on ISBNs, order IDs, and arithmetic examples that resemble card numbers. SSN patterns cover dash-separated, space-separated, and unseparated formats.

Low-risk PII such as emails and phone numbers is redacted rather than blocked. The raw value is replaced with a placeholder such as `[REDACTED_PII]` before the prompt reaches the model. High-risk PII (SSN, API keys, credit cards, passwords) is blocked entirely.

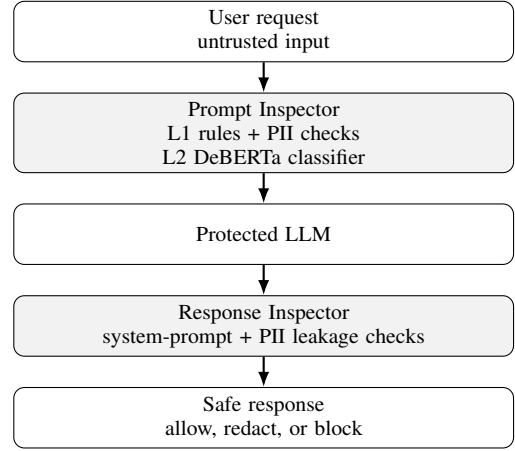


Fig. 1. CheesyWasp performs bidirectional inspection around the protected LLM. Cheap deterministic checks run before classifier-based checks to reduce latency on obvious attacks.

2) *Inbound L2: DeBERTa Classifier:* Inputs that pass L1 are sent to a ProtectAI DeBERTa-v3 small prompt-injection classifier. This layer provides semantic coverage for paraphrased attacks that do not match surface patterns. We use the public weights without fine-tuning and a threshold of 0.5. Cost on CPU is approximately 50–70 ms.

3) *Outbound L1: Response Patterns:* The outbound inspector checks model output for verbatim developer-instruction leakage and PII. Developer-instruction leakage is blocked because it indicates internal-state disclosure. PII in responses is redacted using labelled placeholders such as `[REDACTED_CREDIT_CARD]` to preserve as much response utility as possible.

C. Audit Trail

Every decision is logged with the verdict, inspector, layer, OWASP threat type, human-readable reason, raw model response, and per-stage latency. This structured audit trail supports downstream evaluation and attribution analysis.

IV. EVALUATION

A. Evaluation Suite

We evaluate each threat with a dataset designed for that threat. Table II summarizes the six-track, 749-prompt suite. The protected model is Llama 3.2 1B served via Ollama. We report results for CheesyWasp and a no-firewall pass-through baseline. Prompt Guard 2 was omitted as a standalone comparison because its input-only design cannot evaluate outbound leakage tracks, and a fair comparison on inbound tracks would require running it on the same 749-prompt suite separately from the firewall evaluation, which is left to future work.

B. Metrics

For inbound attack tracks we report three quantities: (a) FW BLOCK RATE, the fraction of malicious prompts blocked by the firewall before reaching the LLM; (b) LLM REFUSE RATE, the fraction that reached the LLM and were refused; and (c)

TABLE II
EVALUATION TRACKS AND DATA SOURCES.

Track	Source	N
Direct prompt injection	Deepset Prompt Injections (English test split)	60
Indirect prompt injection	BIPIA	200
Inbound PII detection	Ai4Privacy PII Masking	200
Benign control / FPR	Dolly-15k	200
System-prompt leakage	Constructed leakage probes	15
Outbound PII leakage	Constructed PII probes	15

TABLE III
MAIN RESULTS ACROSS ALL SIX TRACKS. HANDLED = FIREWALL BLOCKED + LLM REFUSED. FPR MEASURED ON DOLLY-15K BENIGN CONTROL. OUTBOUND TRACKS REPORT LEAK RATE (LOWER IS BETTER); ALL OTHERS REPORT DETECTION OR HANDLING RATE (HIGHER IS BETTER).

Track	FW block	LLM refuse	Handled	FPR
Direct injection ($N=28$ atk)	39.3%	28.6%	67.9%	0.0%
Indirect injection ($N=91$ atk)	8.8%	17.6%	26.4%	4.6%
Inbound PII ($N=200$ atk)	23.5%	58.0%	81.5%	–
Benign control ($N=200$)	–	–	–	0.0%
Sys-prompt leak ($N=15$)	Handled: 100.0% (leak rate: 0.0%)		–	–
Outbound PII ($N=15$)	Handled: 100.0% (leak rate: 0.0%)		–	–

HANDLED RATE = FW block + LLM refuse, the end-to-end fraction that did not produce a compliant response. For benign prompts we report FPR, the fraction of safe prompts incorrectly blocked by the firewall. For outbound tracks we report LEAK RATE, the fraction of probes that produce verbatim leakage of the planted secret in the raw model output. Latency is reported as p50 and p95 of total request time per track.

C. Results

System-prompt leakage. We construct 15 extraction probes paired with a known system prompt containing identifiable strings including a brand name, service name, and operator code. Without the firewall, Llama 3.2 1B leaks at least one identifier on 9 of 15 probes (60%). With CheesyWasp enabled, 0 of 15 probes succeed. All are blocked inbound before the LLM is invoked: 6 at L1 via direct extraction patterns (<1 ms) and 9 at L2 via semantic detection (~55 ms). The response inspector never fires on these probes because the inbound layers intercept the threat upstream, illustrating the cost benefit of ordering cheap layers first.

Outbound PII leakage. We construct 15 probes containing realistic synthetic PII including Luhn-valid credit cards, SSNs, API keys, emails, and phone numbers, and ask the model to echo, summarize, translate, or format them. With CheesyWasp enabled, no probe produces verbatim PII leakage. Twelve are blocked inbound (high-risk PII at L1, semantic injection framing at L2), one is redacted inbound (phone number, low-risk policy), and two reach the LLM but are refused by alignment.

We note a methodological caveat: both outbound probe suites were constructed by the author and test the specific patterns CheesyWasp targets. They demonstrate correct pattern coverage

rather than generalization to novel extraction techniques. Held-out adversarial probes are left to future work.

Direct injection. On the English subset of the Deepset Prompt Injections test split, CheesyWasp blocks 39.3% of attacks at the firewall. The LLM refuses an additional 28.6%, yielding a combined handled rate of 67.9%. The false-positive rate on the benign half is 0.0%, which is notably lower than published FPR numbers for Prompt Guard 2 and Llama Guard on comparable benchmarks [11].

Indirect injection. On BIPIA, CheesyWasp blocks only 8.8% at the firewall and the LLM refuses an additional 17.6%, for a combined handled rate of 26.4%. The high non-compliance rate (73.6%) reflects a fundamental limit of sentence-level classifiers on this benchmark: BIPIA payloads are embedded inside long benign-looking RAG contexts (email bodies, web pages, documents). The DeBERTa classifier attention distributes over hundreds of tokens of benign text, causing the malicious instruction signal to be attenuated. This finding is consistent with Yi et al. [3], who show that LLM-integrated applications with long contexts are highly susceptible to indirect injection. The FPR on BIPIA’s benign portion is 4.6%, driven by the same surface-pattern sensitivity documented by Li et al. [11].

Inbound PII. The combined handled rate is 81.5%: the firewall directly blocks or redacts 23.5% and the LLM refuses an additional 58.0%. The firewall’s 23.5% direct block rate reflects the dataset’s coverage of 54 PII categories, most of which (name, job title, city, DOB, IBAN, GPS coordinate) are outside CheesyWasp’s targeted PII classes. On the seven targeted types (credit card, SSN, API key, password, email, phone, IMEI), block rates are substantially higher: SSN reaches 71.4% after adding multi-format pattern coverage (dash-separated, space-separated, and no-separator variants). Credit card detection stays at 28.6% because the Ai4Privacy dataset uses synthetic card numbers that fail the Luhn checksum, which is the correct and intentional behavior.

False-positive control. CheesyWasp blocks 0.0% of 200 benign Dolly-15k instructions. This is a key result: published guardrail evaluations consistently report non-zero FPR on Dolly-style benign prompts due to trigger-word over-defense [11]. The Luhn verification step is the primary contributor here, preventing false positives on arithmetic expressions and ISBN-shaped strings that a naive 13–19 digit credit-card regex would flag.

V. DISCUSSION AND LIMITATIONS

Scope is the point. CheesyWasp does not attempt to replace the model’s content-policy behavior. Its role is to defend the application boundary against instruction hijacking, sensitive-data exposure, and system-prompt leakage. This is a defense-in-depth design: the firewall and model alignment address complementary failure modes, and the combined handling rate is the correct metric for measuring joint system protection.

Why indirect injection is hard. The 26.4% combined handling rate on BIPIA is honest and consistent with the literature. Sentence-level classifiers lose the injection signal when it is diluted in hundreds of tokens of benign context. Runtime

guardrails are not the right solution here: the more promising direction is architectural separation of trusted instructions from untrusted data, as in StruQ [9] and SecAlign [10], both of which require model retraining and are out of scope for a runtime firewall.

Limitations.

- The DeBERTa L2 classifier can over-defend when benign prompts contain injection-adjacent surface patterns, contributing to the 4.6% FPR on BIPIA.
- The outbound probe suites are author-constructed; results demonstrate pattern coverage, not generalization to novel extraction techniques.
- Indirect injection remains the system’s weakest point, with 73.6% of BIPIA attacks reaching a compliant response.
- The response inspector uses regex-only disclosure patterns. Paraphrased system-prompt disclosure is not caught.
- The inbound PII detector covers seven high-risk PII types; the 47 other categories in Ai4Privacy are outside scope.

Future work. Future versions should add an operator-supplied known-secrets list to the response inspector (a string that must never appear in output verbatim), replace outbound disclosure regexes with a lightweight classifier, and support structured-query separation between trusted instructions and untrusted content following StruQ [9].

VI. CONCLUSION

A runtime LLM firewall is most useful when it defends risks the underlying model cannot reliably defend itself: direct and indirect prompt injection, PII handling, and system-prompt leakage. CheesyWasp is scoped to those threats and evaluated on a six-track, 749-prompt suite that separates injection defense from general harmful-content moderation. The most compelling results are the outbound tracks: system-prompt leakage drops from 60% to 0%, and outbound PII echo reaches 0% leak rate, demonstrating that bidirectional inspection provides meaningful protection where model alignment alone does not. Indirect injection at 26.4% combined handling is the system’s honest limitation and correctly motivates architectural defenses rather than runtime classifier improvement.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [2] Y. Liu *et al.*, “Prompt injection attack against LLM-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2024.
- [3] J. Yi, Y. Xie, B. Zhu, K. Hines, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” *arXiv preprint arXiv:2312.14197*, 2023.
- [4] deepset, “Prompt injections dataset,” HuggingFace, 2023. [Online]. Available: <https://huggingface.co/datasets/deepset/prompt-injections>
- [5] Ai4Privacy, “PII-Masking-200k dataset,” HuggingFace, 2023. [Online]. Available: <https://huggingface.co/datasets/ai4privacy/pii-masking-200k>
- [6] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Analyzing leakage of personally identifiable information in language models,” *IEEE Symposium on Security and Privacy*, 2023. arXiv:2302.00539.
- [7] H. Inan *et al.*, “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” *arXiv preprint arXiv:2312.06674*, 2023.

- [8] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” *arXiv preprint arXiv:2310.10501*, 2023.
- [9] S. Chen *et al.*, “StruQ: Defending against prompt injection with structured queries,” *arXiv preprint arXiv:2402.06363*, 2024.
- [10] S. Chen, A. Zharmagambetov, S. Mahlouiifar, K. Chaudhuri, and C. Guo, “Aligning LLMs to be robust against prompt injection,” *arXiv preprint arXiv:2410.05451*, 2024.
- [11] H. Li *et al.*, “InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models,” *arXiv preprint arXiv:2410.22770*, 2024.
- [12] OWASP Foundation, “OWASP Top 10 for LLM Applications, 2025 edition,” 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>